



Styling Tool Documentation:

Author: Contractor, Daniel

Last update: 2025-02-13

Tool need and purpose

As part of making our work in the IDI more effective, SIA has developed a library of concepts that we have defined during past analysis. When new projects require the same concept, the code that creates a concept can be reused, saving significant development time.

Before new definitions are added to the library, work is required to ensure they are in a consistent format. To save time reworking files we have developed a tool that can automatically apply a consistent style to SQL files.

The tool makes presentation changes to code by changing line breaks, spacing, location of commas, indentation, and capitalisation. By doing these in a consistent manner, the structure of the code becomes clearer and easy to read.

While there are existing tools that serve this purpose (for example the online tool by red-gate.com), none of these are available within the Stats NZ data lab. Online tools are not preferred as (1) they are difficult to batch process, and (2) they require uploading code to the internet.

Example Use

Below is a before and after example of SQL code that has been standardised by the tool. The style we have chosen is based on the 'commas before' style by red-gate.com.

Before

```
select [quarter], [child_snz_uid], max(correction_flag) as corr_flg,  
max(incarcerated_flag) as inc_flg into #children_quarters from  
#source_table
```

After

```
SELECT [quarter]  
      , [child_snz_uid]  
      , MAX(correction_flag) AS corr_flg  
      , MAX(incarcerated_flag) AS inc_flg  
INTO #children_quarters  
FROM #source_table
```

Tool components

The styling tool is available in two formats:

1. A single file version for ease of use. We recommend that most users download and use the single file version. Instructions for execution are provided at the top of this file. No R knowledge is necessary.
2. A multi-file for ease of development. This version splits the tool across several files for ease of reading and updating. It also includes unit tests for confirming correct performance.

Intended use of tool:

The tool is initiated by running the 'Main' code file referred to above, which will prompt the user to choose between the two modes of operation:

- **Single file input:** You can select a single SQL file for processing. The tool will apply the standardization process to this file which will have a "styled_" prefix added to its name to differentiate it from the original.
- **Multiple file input:** You can also select an entire folder of SQL files for processing. The tool will process each file in the folder individually. Please note that the input folder must only contain .sql files (nested subfolders are not allowed).

For both modes, the tool organizes the output into two dedicated subfolders:

- **Unstyled output folder:** A copy of the original, unformatted file(s) is saved in a subfolder named "unstyled", ensuring that you always have access to the unmodified version of the SQL code.
- **Styled output folder:** The processed, formatted file(s) are saved in a subfolder named "styled".

Note R package dependencies: tool should work fine within Datalab – please ensure the following packages are installed and up-to-date if used outside of lab: stringr and dplyr.

Protection method

Styling involves making changes to the layout and presentation of the code. However, we may not want to change some parts of each file (for example, we want to capitalise keywords where they appear in code, but not where they are part of descriptive documentation). This is done by protecting text we do not want to change with placeholders.

Placeholders are used to temporarily replace elements like comments, functions, or custom strings with unique markers in SQL code. This is done to ensure these elements remained unaffected during processing while the rest of the code is styled. During processing these unique markers, such as `<><>cN<><>` for comments or `<><>fN<><>` for functions, ensure these elements remain unchanged while transformations like formatting or capitalization are applied to the rest of the code. After processing, the placeholders are replaced with their original content, restoring the protected code into the changed text.

Justification of logical order

The tool modifies the layout of code during processing. Each of these changes is implemented as a separate function. The ordering of the functions is set out in the function *process_sql_files*:

1. **Protect Blank lines:** Blank lines in the SQL string are identified and replaced with placeholders.
2. **Protect Comments:** Comments are detected and replaced with placeholders.
3. **Capitalize Keywords:** SQL keywords are converted to uppercase.
4. **Protect Functions:** Function calls are identified and replaced with placeholders.
5. **Protect Special Patterns:** Special patterns in the SQL code are replaced with placeholders.
6. **Standardisation:** This removes all newlines and sequences of more than one whitespace
7. **Insert Newlines:** Newlines are added before keywords and brackets for better formatting.
8. **Adjust Indentation:** Proper indentation is applied to the SQL code for readability.
9. **Re-insert spacing:** This restores the whitespaces around special characters such as '>' and '('.
10. **Reassemble Patterns:** Protected special patterns are restored to their original forms.
11. **Reassemble Functions:** Protected functions are restored to their original forms.
12. **Reassemble Comments:** Comments are restored to their original positions.
13. **Remove Extra Newlines:** Any unnecessary double newlines are removed.
14. **Restore Blank lines:** Blank lines are restored to their original positions.

Note that reassembling follows the reverse order of disassembly to ensure logical flow and consistency.

The order of these functions is crucial to ensure proper formatting and restoration of the SQL file. Key reasons for the existing order are as follows:

Preserving blank line placement

To ensure newlines are preserved correctly, blank lines must be protected before comments. If comments are protected first, their placeholders can absorb surrounding blank lines, leading to extra or missing newlines during reassembly.

Consider the following example input code:

```
-- This is a single-line comment

SELECT column1, column2
```

If the logic is performed in the order of protecting comments before blank lines, instead of getting the desired output:

```
-- This is a single-line comment

SELECT column1, column2
```

We get the following incorrect output:

```
-- This is a single-line comment

SELECT column1, column2
```

Which contains an extra blank line.

Avoid capitalizing comments

The *capitalisation* function comes after *protecting comments* but before the *protect functions* and *protect custom strings*. This is to ensure contents of functions and custom strings are capitalised, but the contents of comments are not.

Consider the following example input code:

```
-- This is a comment containing SQL keywords such as where & select  
which should not be capitalised
```

If the logic is performed in the order of performing capitalisation before protecting comments, instead of getting the desired unchanged output, we get the following incorrect output:

```
-- This is a comment containing SQL keywords such as WHERE & SELECT  
which should not be capitalised
```

Which contains unintended capitalisation of keywords.

Arbitrary order of function and pattern protection

The ordering of *Protect Functions* relative to *Replace Special Patterns* is arbitrary: if the functions contain special patterns within them, or vice versa, the contents will be preserved either way.

Standardization before newline insertion and indentation

Standardization, *inserting newlines*, and *applying indentation* must occur in this order to ensure proper formatting of SQL code. During *standardization*, all original newlines are removed to create a uniform structure. Newlines are then reinserted before keywords, brackets, and other structural elements to establish the code's logical flow. Finally, indentation is added at the start of the newly added lines, ensuring each line is indented correctly according to its nesting level or context.

Reinserting spacing before reassembly

Re-insert spacing is done before reassembling the functions and special patterns to avoid adding extra whitespace within the functions or special patterns.

Consider the following example input code:

```
SELECT column1, CONCAT <><>f1<><>
```

If the logic is performed in the order of reassembling functions before re-inserting whitespace, instead of getting the desired output:

```
SELECT column1, CONCAT('A > B' , column2)
```

We get the following incorrect output:

```
SELECT column1, CONCAT ('A > B' , column2)
```

Which contains extra whitespace characters around the '>' and ',' symbols.

Common sources of error

If the tool does not work as expected, begin by checking the following requirements:

- You cannot choose the same input folder as output folder.

- Only files with a '.sql' extension are processed by the tool.
- You must have permission to write to the output folder, but the input folder can be read only.

If the code contains custom SQL keywords that we have not accounted for, this might result in styling inconsistencies in the code. This can be corrected by adding the custom keywords to the keywords list at the top of the functions file.

How to debug

The function *process_sql_files* calls all the functions for the styling tool in the order of the functions above. This provides a useful debugging way by insert breaks after each line in that function. This helps isolate the bug to a specific function of the styling tool.

Please note when debugging that most of the functions return a list containing the SQL code as the first element, and the remaining elements being the placeholder contents. Often it is useful to examine these contents when debugging to ensure the tool is working as expected.

Please also note that the file *styler_unit_tests* contains useful tests for the different functions, which are designed to be simple demonstrations of correctness and hence can provide an easier way to identify if something has gone wrong.